LAB SHEET – 07

Flutter Application Using SQLite (Programs 1–17)

Note: These are compact practical-record programs. Programs 3–17 use the common database setup below. Run each program in a Flutter project and call its function from a screen or main method as needed.

Required packages

```
dependencies:
  flutter:
    sdk: flutter
  sqflite: ^2.4.2
  path: ^1.9.1

Run: flutter pub get
```

Common imports and database helper (Programs 3–17)

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<Database> openDB() async {
  final path = join(await getDatabasesPath(), 'student.db');
  return openDatabase(path, version: 1,
    onCreate: (db, version) async {
      await db.execute(
        'CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, '
        'name TEXT, roll INTEGER, course TEXT)');
    });
}
```

For Programs 1 and 2, use the separate code shown below because they demonstrate database and table creation individually.

1. Create an SQLite database

```
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final path = join(await getDatabasesPath(), 'student.db');
  await openDatabase(path, version: 1);
  runApp(const MaterialApp(
    home: Scaffold(body: Center(
      child: Text('Database Created Successfully'))))));
}
```

Sample Output:
Database Created Successfully

2. Create the students table

```
Future<void> createTable() async {
  final db = await openDB();
  await db.execute(
    'CREATE TABLE IF NOT EXISTS students('
    'id INTEGER PRIMARY KEY AUTOINCREMENT, '
    'name TEXT, roll INTEGER, course TEXT)');
  print('Students Table Created');
}
```

Sample Output:
Students Table Created

3. Insert a record using insert()

```
Future<void> insertStudent() async {
  final db = await openDB();
  await db.insert('students', {
    'name': 'Rahul', 'roll': 101, 'course': 'BCA'
  });
  print('Record Inserted');
}
```

Sample Output:
Record Inserted

4. Insert a record entered through TextFields

```
final name = TextEditingController();
final roll = TextEditingController();

ElevatedButton(
  onPressed: () async {
    await (await openDB()).insert('students', {
      'name': name.text,
      'roll': int.parse(roll.text),
      'course': 'BCA'
    });
    print('Record Inserted');
  },
  child: const Text('Insert'),
);
// Add two TextFields using name and roll as controllers.
```

Sample Output:
Name: Rahul Roll: 101 Course: BCA\nRecord Inserted

5. Insert a record using rawInsert()

```
Future<void> rawInsertStudent() async {
  final db = await openDB();
  await db.rawInsert(
    'INSERT INTO students(name, roll, course) VALUES(?, ?, ?)',
    ['Aman', 102, 'BCA']);
  print('Record Inserted');
}
```

Sample Output:
Record Inserted

6. Display records using query() and ListView

```
class StudentList extends StatelessWidget {
  const StudentList({super.key});

  Future<List<Map<String, dynamic>>> getData() async =>
      (await openDB()).query('students');

  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: getData(),
      builder: (context, snapshot) {
        if (!snapshot.hasData) {
          return const CircularProgressIndicator();
        }
        return ListView(
          children: snapshot.data!.map((s) => ListTile(
            title: Text(s['name'].toString()),
            subtitle: Text(s['course'].toString()),
          )).toList(),
        );
      },
    );
  }
}
```

Sample Output:
Rahul — BCA\nAman — BCA

7. Display records using rawQuery()

```
Future<void> displayRaw() async {
  final db = await openDB();
  final data = await db.rawQuery('SELECT * FROM students');
  print(data);
```

```
}
```

Sample Output:
[{id: 1, name: Rahul, roll: 101, course: BCA}]

8. Update a student using update()

```
Future<void> updateStudent() async {
  final db = await openDB();
  await db.update('students', {'name': 'Amit'},
      where: 'roll = ?', whereArgs: [101]);
  print('Record Updated');
}
```

Sample Output:
Record Updated

9. Delete a student using delete()

```
Future<void> deleteStudent() async {
  final db = await openDB();
  await db.delete('students',
      where: 'roll = ?', whereArgs: [101]);
  print('Record Deleted');
}
```

Sample Output:
Record Deleted

10. Delete all records using Clear All

```
ElevatedButton(
  onPressed: () async {
    await (await openDB()).delete('students');
    print('All Records Deleted');
  },
  child: const Text('Clear All'),
)
```

Sample Output:
All Records Deleted

11. Search a student by roll number

```
Future<void> searchStudent(int roll) async {
  final db = await openDB();
  final data = await db.query('students',
      where: 'roll = ?', whereArgs: [roll]);
  print(data);
}
```

Sample Output:
[{id: 1, name: Rahul, roll: 101, course: BCA}]

12. Display students using DropdownButton

```
String course = 'BCA';

DropdownButton<String>(
  value: course,
  items: ['BCA', 'MCA', 'BBA'].map((c) =>
    DropdownMenuItem(value: c, child: Text(c))).toList(),
  onChanged: (value) async {
    if (value == null) return;
    course = value;
    final data = await (await openDB()).query(
      'students', where: 'course = ?', whereArgs: [course]);
    print(data);
  },
)
```

Sample Output:
Selected Course: BCA\nRahul — BCA

13. Display names in ascending order

```
Future<void> sortStudents() async {
  final data = await (await openDB()).query(
      'students', orderBy: 'name ASC');
  print(data);
```

```
}
```

Sample Output:
Aman\nRahul\nRohit

14. Count total student records

```
Future<void> countStudents() async {
  final db = await openDB();
  final count = Sqflite.firstIntValue(
      await db.rawQuery('SELECT COUNT(*) FROM students'));
  print('Total Students: $count');
}
```

Sample Output:
Total Students: 3

15. Student model with toMap() and fromMap()

```
class Student {
  final int? id;
  final String name, course;
  final int roll;

  Student(this.name, this.roll, this.course, {this.id});

  Map<String, dynamic> toMap() => {
    'name': name, 'roll': roll, 'course': course
  };

  factory Student.fromMap(Map<String, dynamic> map) =>
      Student(map['name'], map['roll'], map['course'],
          id: map['id']);
}

Future<void> saveStudent() async {
  final db = await openDB();
  await db.insert('students',
      Student('Rahul', 101, 'BCA').toMap());
  final rows = await db.query('students');
  final students = rows.map(Student.fromMap).toList();
  print(students.first.name);
}
```

Sample Output:
Rahul

16. DatabaseHelper class

```
class DatabaseHelper {
  Future<int> insert(Map<String, dynamic> data) async =>
      (await openDB()).insert('students', data);

  Future<List<Map<String, dynamic>>> read() async =>
      (await openDB()).query('students');

  Future<int> update(int id, Map<String, dynamic> data) async =>
      (await openDB()).update('students', data,
          where: 'id = ?', whereArgs: [id]);

  Future<int> delete(int id) async =>
      (await openDB()).delete('students',
          where: 'id = ?', whereArgs: [id]);
}
```

Sample Output:
Insert, read, update and delete operations are available.

17. Student Record Management Application

```
class StudentApp extends StatelessWidget {
  StudentApp({super.key});
  final name = TextEditingController();

  Future<void> add() async {
    await (await openDB()).insert('students', {
      'name': name.text, 'roll': 101, 'course': 'BCA'
    });
  }

  @override
```

```
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: const Text('Student Management')),
    body: Column(children: [
      TextField(controller: name,
          decoration: const InputDecoration(labelText: 'Name')),
      ElevatedButton(onPressed: add, child: const Text('Add')),
      ElevatedButton(onPressed: () async {
        print(await (await openDB()).query('students'));
      }, child: const Text('View')),
      ElevatedButton(onPressed: () async {
        await (await openDB()).update('students',
            {'name': 'Aman'}, where: 'id = ?', whereArgs: [1]);
      }, child: const Text('Update')),
      ElevatedButton(onPressed: () async {
        await (await openDB()).delete('students',
            where: 'id = ?', whereArgs: [1]);
      }, child: const Text('Delete')),
    ]),
  );
}

// Run with: runApp(MaterialApp(home: StudentApp()));
```

Sample Output:

Student Management\nAdd View Update Delete

Important: The common helper creates the students table automatically. Program 2 shows the explicit table-creation command for the lab requirement. For Program 4, place the TextFields inside a StatefulWidget or StatelessWidget and declare the controllers. For Program 12, use a StatefulWidget and call setState() after changing the selected course to refresh the dropdown.